In [4]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

In [7]:
```
df=pd.read_csv("C:\Users\System05\Downloads\house.csv")
```

In [8]:
```
df
```

Out[8]:

	crim	zn	indus	chas	nox	rm	age	dis	rad	tax	ptratio	b	lstat
0	0.00632	18.0	2.31	0	0.538	6.575	65.2	4.0900	1	296	15.3	396.90	4.98
1	0.02731	0.0	7.07	0	0.469	6.421	78.9	4.9671	2	242	17.8	396.90	9.14
2	0.02729	0.0	7.07	0	0.469	7.185	61.1	4.9671	2	242	17.8	392.83	4.03
3	0.03237	0.0	2.18	0	0.458	6.998	45.8	6.0622	3	222	18.7	394.63	2.94
4	0.06905	0.0	2.18	0	0.458	7.147	54.2	6.0622	3	222	18.7	396.90	5.33
...	...	...	...	...	...	...	...	...	...	...	...	...	...
501	0.06263	0.0	11.93	0	0.573	6.593	69.1	2.4786	1	273	21.0	391.99	9.67
502	0.04527	0.0	11.93	0	0.573	6.120	76.7	2.2875	1	273	21.0	396.90	9.08
503	0.06076	0.0	11.93	0	0.573	6.976	91.0	2.1675	1	273	21.0	396.90	5.64
504	0.10959	0.0	11.93	0	0.573	6.794	89.3	2.3889	1	273	21.0	393.45	6.48
505	0.04741	0.0	11.93	0	0.573	6.030	80.8	2.5050	1	273	21.0	396.90	7.88

506 rows × 14 columns

In [9]:
```
df.info
```

Out[9]:
```
<bound method DataFrame.info of         crim    zn  indus  chas    nox     rm   ag
e     dis  rad  tax  \
0    0.00632  18.0   2.31     0  0.538  6.575  65.2  4.0900    1  296
1    0.02731   0.0   7.07     0  0.469  6.421  78.9  4.9671    2  242
2    0.02729   0.0   7.07     0  0.469  7.185  61.1  4.9671    2  242
3    0.03237   0.0   2.18     0  0.458  6.998  45.8  6.0622    3  222
4    0.06905   0.0   2.18     0  0.458  7.147  54.2  6.0622    3  222
..       ...   ...    ...   ...    ...    ...   ...     ...  ...  ...
501  0.06263   0.0  11.93     0  0.573  6.593  69.1  2.4786    1  273
502  0.04527   0.0  11.93     0  0.573  6.120  76.7  2.2875    1  273
503  0.06076   0.0  11.93     0  0.573  6.976  91.0  2.1675    1  273
504  0.10959   0.0  11.93     0  0.573  6.794  89.3  2.3889    1  273
505  0.04741   0.0  11.93     0  0.573  6.030  80.8  2.5050    1  273

     ptratio       b  lstat  medv
0       15.3  396.90   4.98  24.0
1       17.8  396.90   9.14  21.6
2       17.8  392.83   4.03  34.7
3       18.7  394.63   2.94  33.4
4       18.7  396.90   5.33  36.2
..       ...     ...    ...   ...
501     21.0  391.99   9.67  22.4
502     21.0  396.90   9.08  20.6
503     21.0  396.90   5.64  23.9
504     21.0  393.45   6.48  22.0
505     21.0  396.90   7.88  11.9

[506 rows x 14 columns]>
```

In [10]:
```
df.shape
```

Out[10]:
```
(506, 14)
```

In [11]:
```
df.columns
```

Out[11]:
```
Index(['crim', 'zn', 'indus', 'chas', 'nox', 'rm', 'age', 'dis', 'rad', 'tax',
       'ptratio', 'b', 'lstat', 'medv'],
      dtype='object')
```

In [27]:
```
df.describe()
```

Out[27]:

	crim	zn	indus	nox	rm	age	dis
count	506.000000	506.000000	506.000000	506.000000	506.000000	506.000000	506.000000
mean	3.613524	11.363636	11.136779	0.554695	6.284341	68.574901	3.795043
std	8.601545	23.322453	6.860353	0.115878	0.702085	28.148861	2.105710
min	0.006320	0.000000	0.460000	0.385000	3.561000	2.900000	1.129600
25%	0.082045	0.000000	5.190000	0.449000	5.885500	45.025000	2.100175
50%	0.256510	0.000000	9.690000	0.538000	6.210000	77.500000	3.207450
75%	3.677083	12.500000	18.100000	0.624000	6.618750	94.075000	5.188425
max	88.976200	100.000000	27.740000	0.871000	8.780000	100.000000	12.126500

In [28]:
```
df.isnull().sum()
```

Out[28]:
```
crim       0
zn         0
indus      0
nox        0
rm         0
age        0
dis        0
rad        0
tax        0
ptratio    0
b          0
lstat      0
medv       0
dtype: int64
```

In [29]:
```
df = df.fillna(df.mean())
```

In [30]:
```
X = df.drop("medv", axis=1)
y = df["medv"]
df
```

Out[30]:

	crim	zn	indus	nox	rm	age	dis	rad	tax	ptratio	b	lstat	med
0	0.00632	18.0	2.31	0.538	6.575	65.2	4.0900	1	296	15.3	396.90	4.98	24.
1	0.02731	0.0	7.07	0.469	6.421	78.9	4.9671	2	242	17.8	396.90	9.14	21.
2	0.02729	0.0	7.07	0.469	7.185	61.1	4.9671	2	242	17.8	392.83	4.03	34.
3	0.03237	0.0	2.18	0.458	6.998	45.8	6.0622	3	222	18.7	394.63	2.94	33.
4	0.06905	0.0	2.18	0.458	7.147	54.2	6.0622	3	222	18.7	396.90	5.33	36.
...	...	...	...	...	...	...	...	...	...	...	...	...	.
501	0.06263	0.0	11.93	0.573	6.593	69.1	2.4786	1	273	21.0	391.99	9.67	22.
502	0.04527	0.0	11.93	0.573	6.120	76.7	2.2875	1	273	21.0	396.90	9.08	20.
503	0.06076	0.0	11.93	0.573	6.976	91.0	2.1675	1	273	21.0	396.90	5.64	23.
504	0.10959	0.0	11.93	0.573	6.794	89.3	2.3889	1	273	21.0	393.45	6.48	22.
505	0.04741	0.0	11.93	0.573	6.030	80.8	2.5050	1	273	21.0	396.90	7.88	11.

506 rows × 13 columns

In [31]:
```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y,test_size=0.2,random_state
```

In [37]:
```
from sklearn.linear_model import LinearRegression
model = LinearRegression()
model.fit(X_train, y_train)
```

Out[37]:
```
▾ LinearRegression
▶ Parameters
```

In [38]:
```
y_train_pred = model.predict(X_train)
y_test_pred = model.predict(X_test)
```

In [39]:
```
from sklearn.metrics import mean_squared_error, r2_score
import numpy as np
```

In [40]:
```
mse = mean_squared_error(y_test, y_test_pred)
print("MSE:", mse)
```

```
MSE: 24.746850670022326
```

In [41]:
```
rmse = np.sqrt(mse)
print("RMSE:", rmse)
```

```
RMSE: 4.974620655891495
```

In [42]:
```
r2 = r2_score(y_test, y_test_pred)
print("R2 Score:", r2)
```

```
R2 Score: 0.6625450153591483
```

In [44]:
```
y_mean = np.mean(y_test)
y_mean
```

Out[44]:
```
np.float64(21.488235294117644)
```

In [45]:
```
SSE = np.sum((y_test - y_test_pred)**2)
print("SSE:", SSE)
```

```
SSE: 2524.178768342277
```

In [46]:
```
SST = np.sum((y_test - y_mean)**2)
print("SST:", SST)
```

```
SST: 7480.045882352942
```

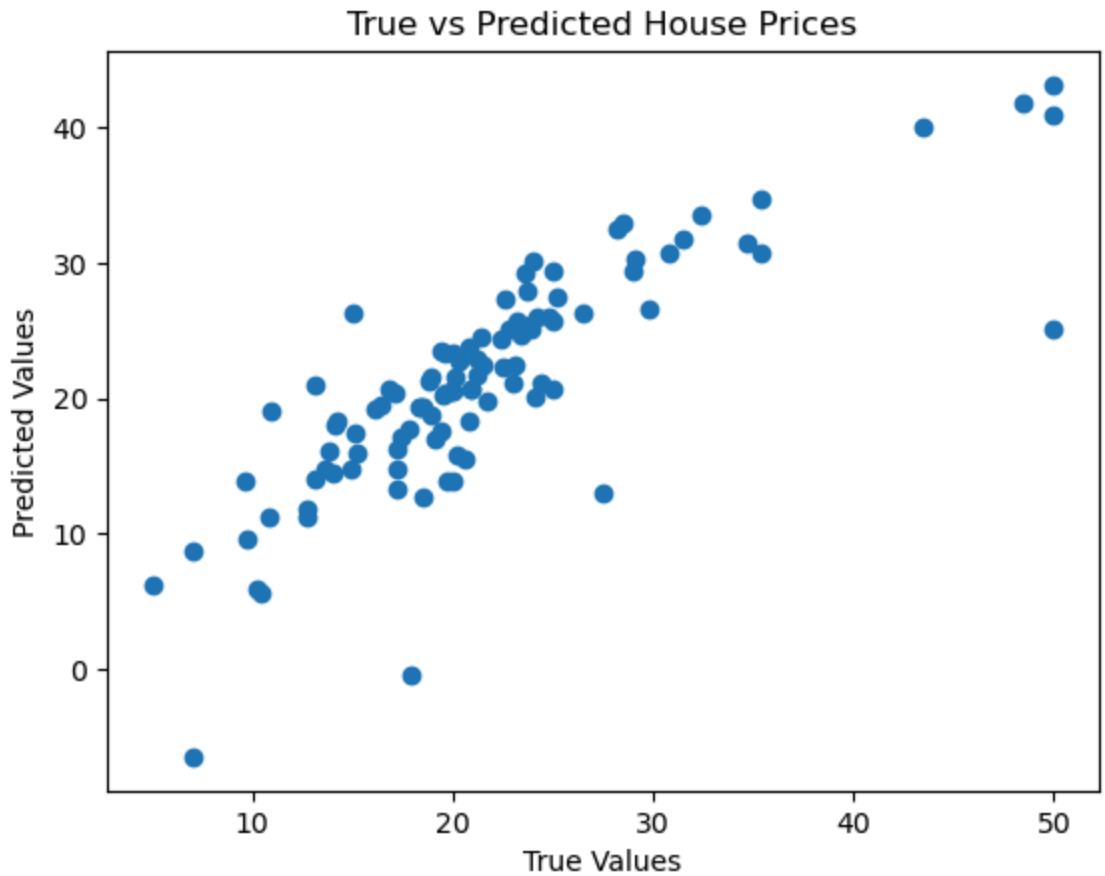
In [47]:
```
SSR = SST - SSE
print("SSR:", SSR)
```

```
SSR: 4955.867114010664
```

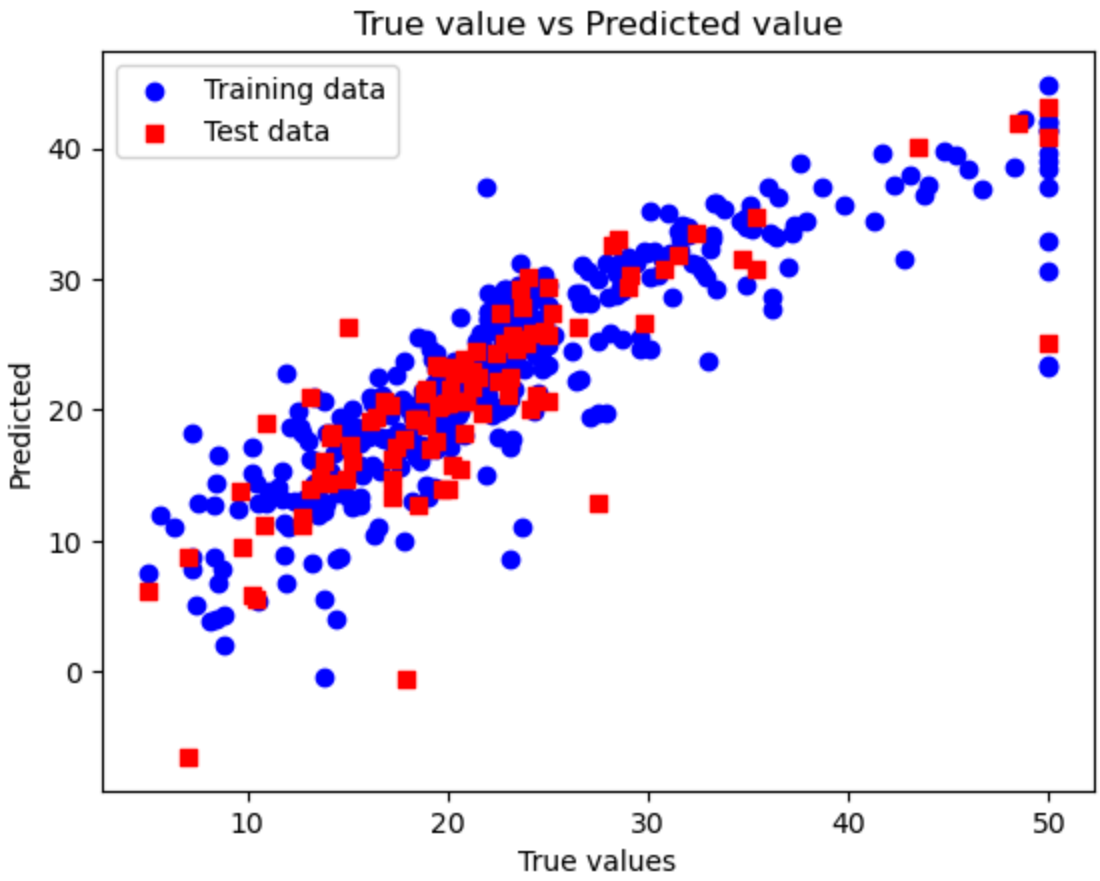
In [48]:
```
r2_manual = SSR / SST
print("Manual R2:", r2_manual)
```

```
Manual R2: 0.6625450153591484
```

In [49]:
```
plt.scatter(y_test, y_test_pred)
plt.xlabel("True Values")
plt.ylabel("Predicted Values")
plt.title("True vs Predicted House Prices")
plt.show()
```



In [53]:
```
plt.scatter(y_train, ytrain_pred,c='black',marker='o',label='Training data')
plt.scatter(y_test, ytest_pred,c='red',marker='s',label='Test data')
plt.xlabel('True values')
plt.ylabel('Predicted')
plt.title("True value vs Predicted value")
plt.legend(loc='upper left')
plt.plot()
plt.show()
```


True value vs Predicted value
Training data
Test data
Predicted
True values
0
10
20
30
40
10
20
30
40
50

In [ ]: